

SmartRecruitz

Technical Design Document

Table of Contents

1. Scope & Objectives	
2. Tech Stack	
3. Candidate Pipeline Flow	
4. Agent Breakdown — All 5 Agents	
5. Architecture — 3-Layer Design	
6. Database Schema — 11 Tables	
7. API Endpoints	
8. Candidate Status Flow	

1 Scope & Objectives

What we're building and what we're not building in this phase.

Objective

Build 5 AI-powered agents that process candidates from resume upload to verified talent pool entry. Each agent runs independently, called sequentially through a chatbot-style frontend interface.

In Scope — 5 Agents

#	AGENT	PURPOSE
1	Resume Parser & Skill Normalization	Parse resume, extract structured data, normalize skills to taxonomy
2	Duplicate Detection	Detect existing profiles using DB + AI semantic matching
3	ID Verification	Verify identity documents against claimed data
4	Interview Question Generation	Generate personalized interview questions from candidate profile
5	Interview Scoring & Readiness	Score interview transcript, assign readiness score

2 Tech Stack

Technologies and frameworks used across the platform.

LAYER	TECHNOLOGY	RATIONALE
Backend API	FastAPI (Python 3.12)	Async, auto-docs, type-safe, fast
AI Model	Claude Sonnet (Anthropic SDK)	All 5 agents — text + document/image analysis
Database	PostgreSQL	JSONB for flexible data, relational for structured
ORM	SQLAlchemy (async)	Type-safe models, migrations via Alembic
Task Queue	ARQ + Redis	Async background processing (AI calls take 5-15 sec)
Validation	Pydantic	Input/output schema validation
File Storage	Local (dev) / S3 (prod)	Resume, ID document storage
Containers	Docker + Compose	Postgres, Redis, app, worker in one command

Why no LangChain / LangGraph? All agents are single-call to Claude — no chaining, no branching, no AI-controlled flow. The pipeline is linear and code-controlled. Anthropic SDK directly is simpler, faster, and easier to debug.

3 Candidate Pipeline Flow

End-to-end journey from resume upload to talent pool entry.

IN

Candidate Registration

Chatbot collects name, email, phone — saved to database. No AI needed.



A1

Agent 1 — Resume Parser & Skill Normalization

Candidate uploads resume. AI reads PDF/DOCX/Image directly, extracts all fields, normalizes skills to standard taxonomy, identifies domains, infers implied skills. Also does quick duplicate pre-check (email/phone exact match).

Claude Sonnet (with file attached)

→ Saved to **candidates_staging** (status: PENDING)



A2

Agent 2 — Duplicate Detection

Step 1 (DB — free): Query candidates_main for potential matches using email, phone, fuzzy name, company+dates. Narrows thousands → 5-10 candidates.

Step 2 (AI — one call): Claude compares new candidate against 5-10 matches. Semantic name matching (Bob=Robert), timeline validation, confidence scoring.

DB Query + Claude Sonnet

→ UNIQUE: Move to **candidates_main** (status: VERIFIED)

→ DUPLICATE (>75%): Merge or reject

→ UNCERTAIN (50-75%): Flag for HR review



A3

Agent 3 — ID Verification

Candidate uploads ID document (Aadhaar/PAN/Passport). AI reads image, extracts text via OCR, checks for tampering indicators, compares extracted data against claimed registration data.

Claude Sonnet (with image attached)

→ VERIFIED: status → IDENTITY_VERIFIED

→ FAILED: status → VERIFICATION_FAILED

Note: No connection to government databases — internal consistency check only.



A4

Agent 4 — Interview Question Generation

AI analyzes candidate's specific experience, skills, and target role. Generates 10-15 personalized questions that target claimed expertise. Each question is hashed and checked against history to prevent repetition.

Claude Sonnet

→ Questions + expected answers stored in DB. Interviewer guide generated.



VI

Video Interview (External — Interview as Service Module)

Candidate records video answers. External service handles proctoring, face matching, liveness detection. SmartRecruitz receives the transcript + genuineness flags.

Separate Module — Not in this build



A5

Agent 5 — Interview Scoring & Readiness

AI evaluates interview transcript against expected answers from Agent 4. Scores each answer (CORRECT/PARTIAL/INCORRECT), assesses technical knowledge and communication, generates overall readiness score.

Claude Sonnet

→ Score ≥ 60%: status → **TALENT_POOL** 

→ Score < 60%: status → INTERVIEW_FAILED (retry after 30 days)



Verified Talent Pool

Candidate is now verified, interviewed, and scored. Visible to hiring managers for job matching (Agent 6 — future phase).

4 Agent Breakdown

Detailed input, output, and AI model for each agent.

1

Resume Parser & Skill Normalization

Claude Sonnet — with PDF/DOCX/Image file attached

Parses unstructured resumes into structured candidate profiles. Normalizes skills to standard taxonomy. Identifies domains per experience entry and calculates domain-wise experience.

INPUT

- Resume file (PDF, DOCX, or Image)
- Candidate reference ID
- Extraction config flags

OUTPUT

- Personal info (name, email, phone, title)
- Experience entries with domain classification
- Education, certifications, projects
- Normalized skills with proficiency levels
- Implied skills with confidence scores
- Domain-wise experience breakdown
- Overall confidence score (0-1)

Quick Duplicate Pre-Check: After parsing, does an exact email/phone match against candidates_main (DB query, no AI). Flags obvious duplicates immediately — saves an AI call.

2

Duplicate Detection

DB Query (free) + Claude Sonnet (one call)

Two-step process to find duplicates without sending all records to AI.

Step 1 — Database Filtering (Free, Milliseconds)

QUERY	WHAT IT FINDS
Email exact match	Same email in candidates_main
Phone exact match (normalized)	Same phone after removing +, -, spaces
Fuzzy name match (pg_trgm)	Similar names (score > 80%)
Company + dates overlap	Same employer in same period

Narrows thousands of records → **5-10 potential matches**.

Step 2 — AI Verification (One Claude Call)

SENT TO AI

- New candidate profile
- 5-10 potential matches from DB
- Confidence threshold (default: 0.75)

AI RETURNS

- Duplicate yes/no + confidence %
- Match evidence (name, phone, experience)
- Semantic analysis (Bob = Robert)
- Recommendation: MERGE / REJECT / REVIEW

Name alone is never enough. AI only flags duplicates when name + at least one other signal match (phone, email, company+dates, education+year).

3 ID Verification

Claude Sonnet — with ID document image attached

Verifies candidate identity by reading uploaded ID documents and comparing against claimed registration data.

INPUT

- ID document image (Aadhaar / PAN / Passport)
- Claimed data from registration (name, DOB, gender)

OUTPUT

- Extracted data from document (OCR)
- Field-by-field match scores (name, DOB, etc.)
- Document authenticity check (tampering indicators)
- Overall verification status + confidence

Important: We do NOT connect to Aadhaar/PAN government databases. We verify **internal consistency** between what the document says and what the candidate claimed during registration.

4 Interview Question Generation

Claude Sonnet

Generates personalized L1 interview questions based on candidate's actual experience and target role.

INPUT

- Candidate profile + skill profile (from Agent 1)
- Experience highlights with responsibilities
- Target role + generic role template
- Question config (count, difficulty, categories)

OUTPUT

- 10-15 personalized questions
- Category per question (Technical / Behavioral / Experience)
- Expected answer points for each question
- Follow-up questions

→ Interviewer guide (focus areas, red flags)

Question Non-Repetition

METHOD	HOW IT WORKS
Question Hashing	Each question is hashed and stored — prevents exact duplicates
Semantic Check	AI compares against recent history — prevents paraphrased duplicates
Experience Targeting	Questions target unique experience — naturally different per candidate
Candidate History	If re-applying, system knows previous questions — never repeats

5 Interview Scoring & Readiness

Claude Sonnet

Analyzes interview transcript and scores each answer against the expected answer points generated by Agent 4.

INPUT

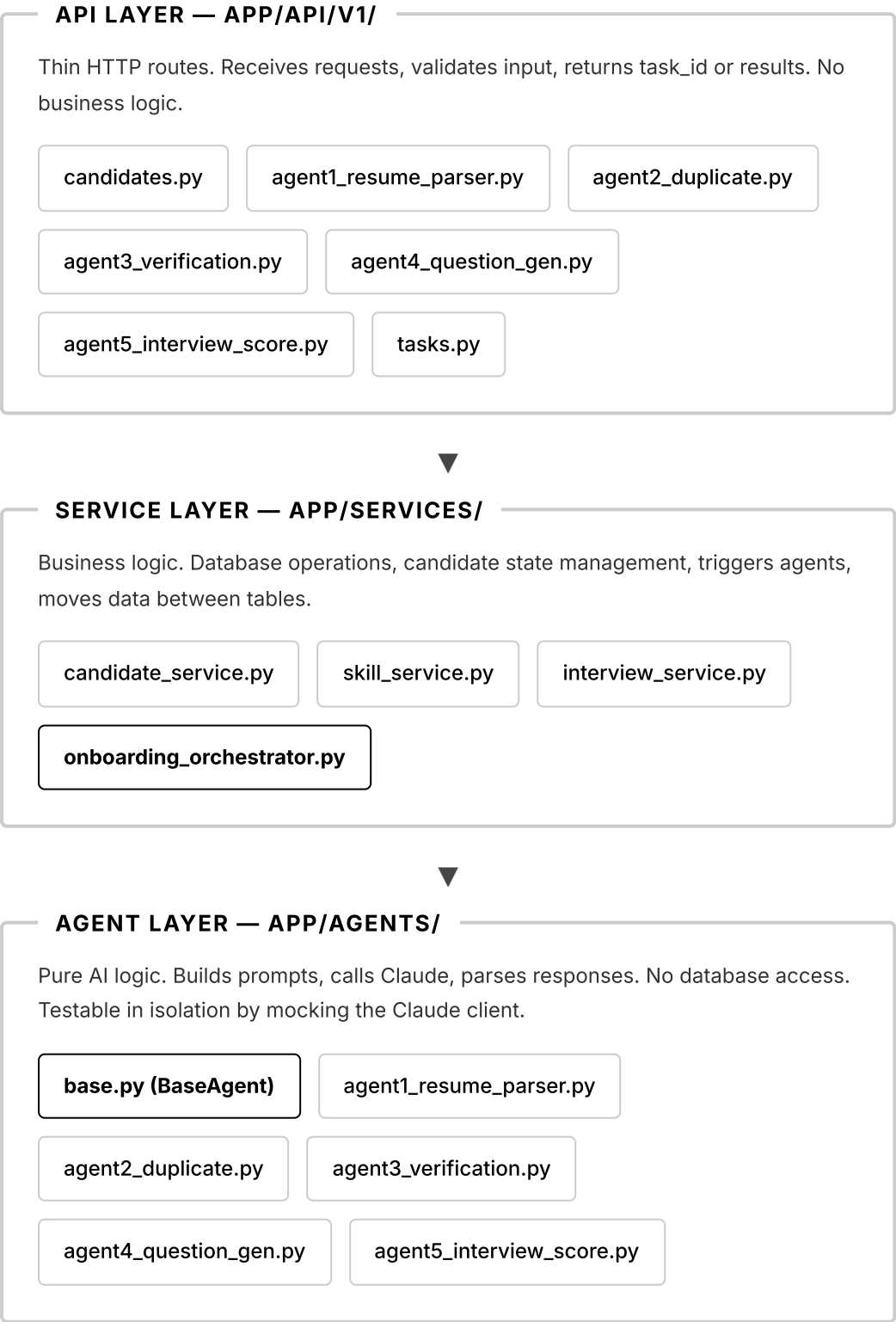
- Interview transcript (from video service)
- Questions + expected answers (from Agent 4)
- Evaluation criteria

OUTPUT

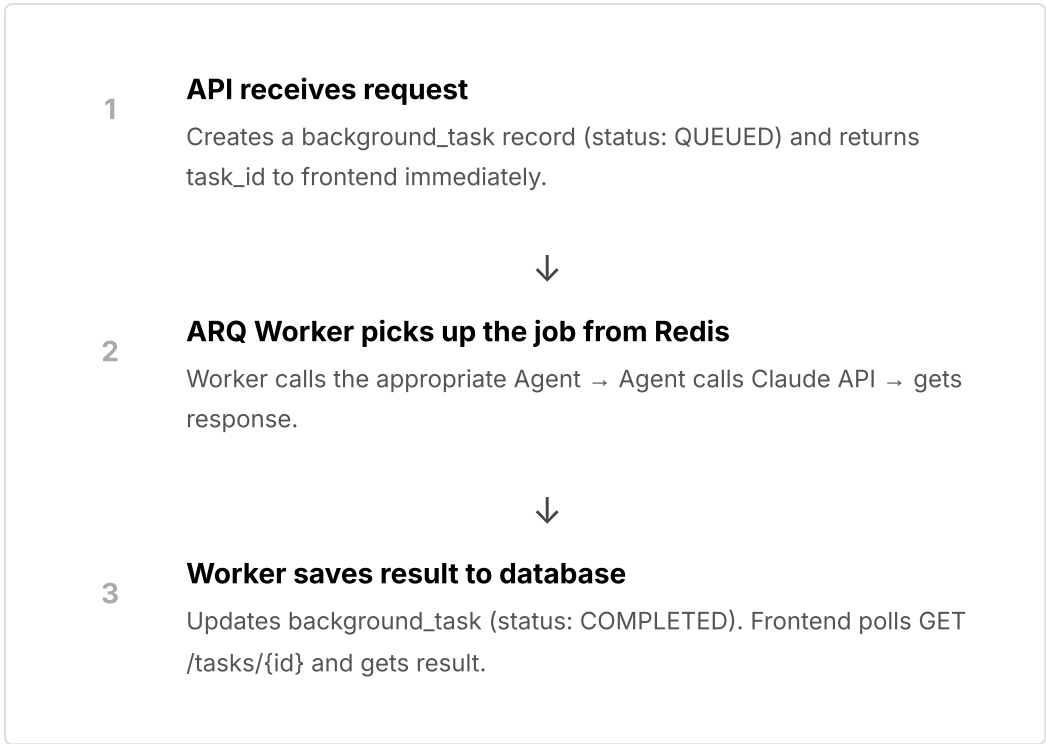
- Overall score (0-100)
- Pass/Fail status (threshold: 60%)
- Per-question score + quality assessment
- Category breakdown (technical, communication, etc.)
- Recommendation (PROCEED / REJECT / REVIEW)
- Readiness level

5 Architecture — 3-Layer Design

Clean separation between HTTP, business logic, and AI processing.



Background Processing Flow



Shared Infrastructure

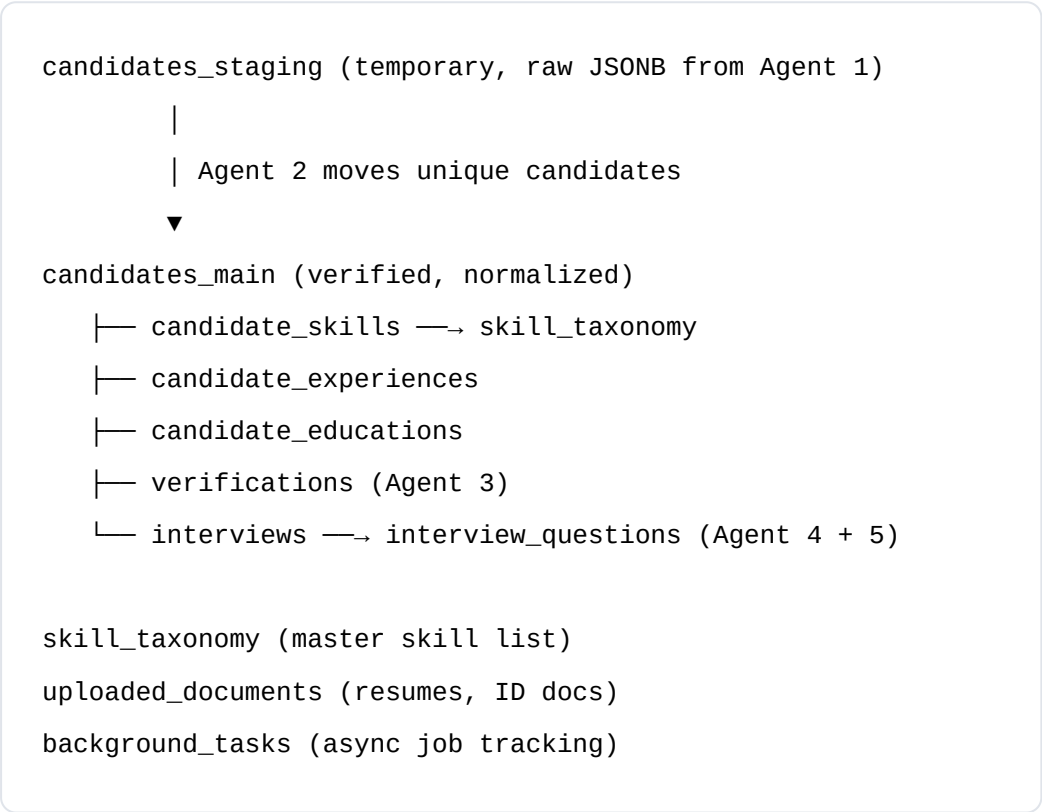
COMPONENT	LOCATION	PURPOSE
Claude Client	app/core/claude_client.py	Wraps Anthropic SDK. send_text() for text agents, send_vision() for file/image agents. Retry logic, token tracking.
File Storage	app/core/file_storage.py	Upload/download files. Local disk (dev) or S3 (prod). get_base64() for sending to Claude Vision.
Task Manager	app/core/background_tasks.py	Enqueues jobs to Redis via ARQ. Creates DB tracking record. Returns task_id.

COMPONENT	LOCATION	PURPOSE
Base Agent	app/agents/base.py	Abstract base class. All 5 agents extend this. Loads prompts from text files.

6 Database Schema — 11 Tables

PostgreSQL database design for the candidate pipeline.

Table Relationships



Design Decision: `candidates_staging` stores raw data as JSONB (not normalized) because it may be rejected or merged. Only when moved to `candidates_main` does data get normalized into child tables.

candidates_staging

Temporary storage for new candidates from Agent 1. Deleted or moved after Agent 2 processes them.

COLUMN	TYPE	DESCRIPTION
id	UUID PK	Auto-generated

COLUMN	TYPE	DESCRIPTION
candidate_ref	VARCHAR(50)	Business key "SR-2024-00123" — unique, indexed
status	VARCHAR(30)	PENDING DUPLICATE_REVIEW AWAITING_HR_REVIEW COMPLETED REJECTED
full_name	VARCHAR(255)	
email	VARCHAR(255)	Indexed for duplicate check
phone	VARCHAR(50)	Indexed for duplicate check
current_title	VARCHAR(255)	
raw_resume_data	JSONB	Full Agent 1 output
skills_normalized	JSONB	Array of normalized skills
experience	JSONB	Array of experience entries
education	JSONB	Array of education entries
certifications	JSONB	
projects	JSONB	
languages	JSONB	
total_experience_years	FLOAT	
primary_domain	VARCHAR(100)	FinTech, Healthcare, E- Commerce, etc.
domain_experience	JSONB	Domain-wise breakdown
parse_confidence	FLOAT	AI confidence 0-1
duplicate_pre_check	JSONB	Quick check result
<u>resume_document_id</u>	UUID FK	→ uploaded_documents
created_at	TIMESTAMP	

COLUMN	TYPE	DESCRIPTION
updated_at	TIMESTAMP	

candidates_main

Verified candidates in the talent pool. Normalized child tables for skills, experience, education.

COLUMN	TYPE	DESCRIPTION
id	UUID PK	
candidate_ref	VARCHAR(50)	Unique business key
status	VARCHAR(30)	VERIFIED IDENTITY_VERIFIED INTERVIEW_READY TALENT_POOL INTERVIEW_FAILED
full_name	VARCHAR(255)	
email	VARCHAR(255)	Unique
phone	VARCHAR(50)	Unique
current_title	VARCHAR(255)	
total_experience_years	FLOAT	
primary_domain	VARCHAR(100)	
location	VARCHAR(255)	
target_role	VARCHAR(255)	Self-declared during registration
readiness_score	FLOAT	Set by Agent 5 (0-100)
verification_status	VARCHAR(30)	PENDING VERIFIED FAILED
talent_pool_entry_date	TIMESTAMP	When they entered talent pool
raw_profile_data	JSONB	Complete parsed profile backup

COLUMN	TYPE	DESCRIPTION
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

candidate_skills

COLUMN	TYPE	DESCRIPTION
id	UUID PK	
<u>candidate_id</u>	UUID FK	→ candidates_main
<u>skill_id</u>	UUID FK	→ skill_taxonomy
proficiency	VARCHAR(20)	BEGINNER INTERMEDIATE ADVANCED EXPERT
years_experience	FLOAT	
evidence	VARCHAR(500)	"4 years building microservices"
is_implied	BOOLEAN	True if inferred by AI
confidence	FLOAT	AI confidence for implied skills

candidate_experiences

COLUMN	TYPE	DESCRIPTION
id	UUID PK	
<u>candidate_id</u>	UUID FK	→ candidates_main
company	VARCHAR(255)	
title	VARCHAR(255)	
start_date	VARCHAR(20)	"2020-01"

COLUMN	TYPE	DESCRIPTION
end_date	VARCHAR(20)	Null if current
duration_months	INT	
domain	VARCHAR(100)	FinTech, Healthcare, etc.
is_current	BOOLEAN	
responsibilities	JSONB	Array of strings

candidate_educations

COLUMN	TYPE	DESCRIPTION
id	UUID PK	
<u>candidate_id</u>	UUID FK	→ candidates_main
institution	VARCHAR(255)	
degree	VARCHAR(100)	
field	VARCHAR(255)	
graduation_year	INT	

skill_taxonomy

Master skill list. All candidate skills must map to this taxonomy.

COLUMN	TYPE	DESCRIPTION
id	UUID PK	
standard_name	VARCHAR(255)	"JavaScript" — unique
category	VARCHAR(100)	Programming Language, Framework, Cloud, etc.
aliases	JSONB	["JS", "ECMAScript", "Javascript"]

COLUMN	TYPE	DESCRIPTION
<u>parent_skill_id</u>	UUID FK	Self-referential for hierarchy

verifications

COLUMN	TYPE	DESCRIPTION
id	UUID PK	
verification_ref	VARCHAR(50)	"VER-2024-78901"
<u>candidate_id</u>	UUID FK	→ candidates_main
verification_type	VARCHAR(20)	IDENTITY
document_type	VARCHAR(30)	AADHAAR_CARD PAN_CARD PASSPORT
status	VARCHAR(20)	PENDING VERIFIED FAILED MANUAL_REVIEW
confidence_score	FLOAT	
extracted_data	JSONB	What AI read from document
data_match_results	JSONB	Field-by-field comparison
document_authenticity	JSONB	Fraud indicators, quality score
flags	JSONB	Any issues found
<u>document_id</u>	UUID FK	→ uploaded_documents
created_at	TIMESTAMP	

interviews

COLUMN	TYPE	DESCRIPTION
id	UUID PK	

COLUMN	TYPE	DESCRIPTION
interview_ref	VARCHAR(50)	"INT-2024-00456"
<u>candidate_id</u>	UUID FK	→ candidates_main
interview_type	VARCHAR(20)	L1_SCREENING
status	VARCHAR(30)	QUESTIONS_GENERATED SCHEDULED COMPLETED SCORED
transcript	TEXT	Full interview transcript
overall_score	FLOAT	Set by Agent 5 (0-100)
l1_status	VARCHAR(20)	PASSED FAILED
recommendation	VARCHAR(30)	PROCEED_TO_L2 REJECT MANUAL_REVIEW
evaluation_data	JSONB	Full Agent 5 scoring output
interviewer_guide	JSONB	Focus areas, red flags to watch
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

interview_questions

COLUMN	TYPE	DESCRIPTION
id	UUID PK	
<u>interview_id</u>	UUID FK	→ interviews
question_ref	VARCHAR(20)	"Q-001"
category	VARCHAR(30)	EXPERIENCE_VERIFICATION TECHNICAL BEHAVIORAL
question_text	TEXT	The actual question

COLUMN	TYPE	DESCRIPTION
targets_skill	VARCHAR(100)	What skill this tests
targets_experience	VARCHAR(255)	What experience claim this verifies
difficulty	VARCHAR(20)	MID SENIOR LEAD
expected_answer_points	JSONB	Array of expected answer points
follow_up_questions	JSONB	
time_estimate_mins	INT	
answer_quality	VARCHAR(20)	CORRECT PARTIAL INCORRECT (set by Agent 5)
answer_score	FLOAT	Set by Agent 5
question_hash	VARCHAR(64)	For deduplication — indexed
created_at	TIMESTAMP	

uploaded_documents

COLUMN	TYPE	DESCRIPTION
id	UUID PK	
filename	VARCHAR(255)	Stored filename
original_filename	VARCHAR(255)	What user uploaded
content_type	VARCHAR(100)	"application/pdf", "image/png"
file_size_bytes	INT	
storage_path	VARCHAR(500)	Path in file storage
document_category	VARCHAR(30)	RESUME ID_DOCUMENT

COLUMN	TYPE	DESCRIPTION
uploaded_by_ref	VARCHAR(50)	candidate_ref
created_at	TIMESTAMP	

background_tasks

COLUMN	TYPE	DESCRIPTION
id	UUID PK	
task_type	VARCHAR(50)	AGENT1_RESUME_PARSE AGENT2_DUPLICATE AGENT3_VERIFY AGENT4_QUESTIONS AGENT5_SCORE
reference_id	VARCHAR(50)	candidate_ref or interview_ref
status	VARCHAR(20)	QUEUED PROCESSING COMPLETED FAILED
result	JSONB	Agent output when completed
error_message	TEXT	If failed
created_at	TIMESTAMP	
started_at	TIMESTAMP	
completed_at	TIMESTAMP	

7 API Endpoints

All endpoints for the candidate pipeline.

Candidate Registration

METHOD	PATH	DESCRIPTION
POST	/api/v1/candidates/register	Save name, email, phone
GET	/api/v1/candidates/staging	List staging candidates
GET	/api/v1/candidates/staging/{ref}	Get staging candidate detail
GET	/api/v1/candidates	List verified candidates
GET	/api/v1/candidates/{ref}	Get full candidate profile

Agent Endpoints

All agent endpoints return a **task_id** immediately (async). Frontend polls /tasks/{id} for result.

METHOD	PATH	AGENT	INPUT
POST	/api/v1/agent1/parse-resume	Resume Parser	multipart file + candidate_ref
POST	/api/v1/agent2/check-duplicate	Duplicate Check	candidate_ref + threshold
POST	/api/v1/agent2/resolve	HR Resolve	candidate_ref + action (MERGE/REJECT/KEEP)
POST	/api/v1/agent3/verify-identity	ID Verification	multipart image + candidate_ref + document_type

METHOD	PATH	AGENT	INPUT
POST	/api/v1/agent4/generate-questions	Question Gen	candidate_ref + interview_type + config
POST	/api/v1/agent5/score-interview	Interview Score	interview_ref + transcript

Task Polling

METHOD	PATH	DESCRIPTION
GET	/api/v1/tasks/{task_id}	Check task status + result when completed

Skill Taxonomy

METHOD	PATH	DESCRIPTION
GET	/api/v1/skills/taxonomy	List all skills (with search)
POST	/api/v1/skills/taxonomy	Add skill to taxonomy

8 Candidate Status Flow

How candidate status changes through the pipeline.

STAGE	TABLE	STATUS	TRIGGERED BY
Resume uploaded	candidates_staging	PENDING	Agent 1

STAGE	TABLE	STATUS	TRIGGERED BY
Possible duplicate	candidates_staging	DUPLICATE_REVIEW	Agent 2
Duplicate confirmed	candidates_staging	REJECTED	Agent 2 / HR
Unique confirmed	candidates_main	VERIFIED	Agent 2
ID verified	candidates_main	IDENTITY_VERIFIED	Agent 3
ID failed	candidates_main	VERIFICATION_FAILED	Agent 3
Questions generated	candidates_main	INTERVIEW_READY	Agent 4
Interview passed	candidates_main	TALENT_POOL	Agent 5
Interview failed	candidates_main	INTERVIEW_FAILED	Agent 5